

Exact Unknot Recognition and the Quasipolynomial Implementation Gap

Executable baseline, certificates, and hierarchy components

17 September 2026

Abstract

The requested deliverable was a total unknot recognizer with running time $n^{O(\log n)}$ for an n -crossing diagram. That implementation has not been completed. This archive instead supplies a complete exact recognizer for rectangular diagrams, based on monotone simplification, together with replayable certificates and implemented polynomial components of the hierarchy approach. Its proved worst-case class is $2^{O(g \log g)}$ in the number g of grid rows, not the requested quasipolynomial class. This report gives the actual algorithms, their correctness arguments, reproducible tests, and the precise constructions and bounds still needed to turn the supplied Lackenby notes into the requested implementation. In particular, no named geometric operation is replaced by an unimplemented oracle or silently charged constant time.

1 Scope and the source specification

The supplied slides [1] announce a quasipolynomial algorithm, then explain a hierarchy construction and simplification process. The relevant objects are knot exteriors, properly embedded surfaces, boundary patterns, multisurfaces, and generalized Heegaard splittings. The four speed-ups on pages 66–74 are a quadratic complexity bound for surfaces, compressed hierarchy encoding, multisurfaces, and Cheeger regions. The final flowchart, page 109, explicitly names the geometric operations needed to combine them.

The additional specification examined was Lackenby’s July 2026 paper [2]. Its Section 9 distinguishes iteration counting from the costs of steps that have not been specified sufficiently to estimate running time; it does not there bound its two parameters G and L . This is a statement about that source and this implementation, not a claim that a full quasipolynomial specification cannot be developed or exists nowhere else.

What the program decides. A rectangular diagram is accepted in the format described in Section 2. The unlimited recognizer always decides whether its single component is unknotted. It is a different algorithm from the flowchart. The hierarchy-related modules are standalone components, not a completed hierarchy engine. Arbitrary PD codes, DT codes, photographs, and polygonal embeddings are not converted automatically.

Three outcomes under limits. Without resource limits, the mathematical algorithm returns UNKNOT or KNOTTED. With a state or time limit it may return UNKNOWN. An incomplete search never becomes a negative answer. Memory exhaustion or external process termination supplies no conclusion.

2 Rectangular diagrams and elementary moves

2.1 Representation and validation

A grid of size $g \geq 2$ has two marked corners in each of g rows and each of g columns. Its storage is an immutable tuple

$$R = ((a_0, b_0), \dots, (a_{g-1}, b_{g-1})), \quad 0 \leq a_i < b_i < g.$$

Rows are ordered bottom to top and columns left to right. Join the two corners in each row horizontally and the two in each column vertically. At every intersection away from corners, the vertical strand passes over the horizontal strand. Thus every validated input defines a classical link diagram; there is no virtual-planarity ambiguity.

The corner incidence graph has one vertex for each row and column, and one edge for each marked corner. Every vertex has degree two. Its connected components are exactly the link components, which can be counted by graph traversal. The recognizer rejects inputs with more than one component. Grid orientation is not retained; either orientation gives the same unknot answer.

An equivalent input consists of two permutations X, O of $\{0, \dots, g-1\}$ with $X_i \neq O_i$: store row i as $\{X_i, O_i\}$. For example,

```
{"x": [0, 1, 2, 3, 4], "o": [2, 3, 4, 0, 1]}
```

is the bundled five-row example with three crossings and determinant three.

2.2 Cyclic equivalence

Cyclically shifting all rows or all columns preserves the represented link. The search identifies grids under these shifts. For each of the g column shifts, it computes the least cyclic rotation of the row tuple using Booth's algorithm; the least resulting tuple is the representative. The cost is $O(g^2)$ small-integer operations. No arbitrary row permutation, reflection, or crossing switch is included in the quotient.

The verifier deliberately uses a different canonicalizer: enumerate all g^2 cyclic shifts and choose the least tuple. Its $O(g^3)$ cost is acceptable because it avoids depending on the optimized canonicalization algorithm.

2.3 Exchanges

For two cyclically adjacent rows with endpoints (a, b) and (c, d) , an exchange is permitted precisely when the endpoint sets are disjoint and they do not alternate on the column circle. With each pair sorted, the forbidden alternating orders are

$$a < c < b < d \quad \text{and} \quad c < a < d < b.$$

The column exchange is the same test with rows and columns reversed. Adjacent pairs include $(g-1, 0)$: a cyclic shift brings this case to ordinary adjacency. Rows sharing an endpoint are *not* accepted by this implementation's exchange rule. These are the exchange moves used in the monotone simplification result [3].

2.4 Destabilizations

Suppose a corner (r, c) has the other endpoint (r, d) in its row and the other endpoint (s, c) in its column. A destabilization is allowed when r, s are cyclically adjacent, c, d are cyclically adjacent,

and (s, d) is absent. The three occupied corners form a small L-shaped bend. Delete row r and column c , replace the bend by (s, d) , and renumber the remaining rows and columns. The operation is forbidden at $g = 2$.

The same rule covers every position and orientation of the little rectangle. Wrap-around cases are conjugate to ordinary cases by cyclic shifts. Both the fast generator and the checked application routine test the full conditions. A separate fixture generator implements inverse stabilizations, but the recognizer itself never increases grid size.

3 The exact recognition algorithm

3.1 Search procedure

The initial grid is canonicalized. A deque holds discovered states and a parent map records one predecessor and one move per state. The optional non-unit determinant obstruction is tried first. In the search itself, all legal exchanges and destabilizations are generated and their results canonicalized. New smaller grids are placed at the front of the deque; other new grids are placed at the back. This changes search order, not the reachable state set. A two-row grid terminates the search positively.

If the deque empties without reaching two rows, the complete discovered set is returned as a negative certificate. No heuristic pruning is used. The recognizer does not assume that a local minimum is nontrivial unless all its allowed successors have been accounted for.

Theorem 3.1 (Correctness and termination). *Assuming the standard monotonic simplification theorem for arc-presentations, the unlimited recognizer terminates and returns `UNKNOT` exactly for the unknot.*

Proof. Every move and every cyclic shift preserves knot type, and the unique two-row grid is an unknot. Hence a successful path gives a correct positive answer. Dynnikov’s Proposition 6 [3] states that an unknot admits a finite sequence of exchanges and destabilizations leading to a trivial presentation. Passing to cyclic-shift representatives preserves this path: all cyclically adjacent move locations are considered. Therefore an unknot cannot remain undetected after the reachable state set is exhausted.

Finally, grid size never exceeds its initial value, and each size has finitely many grids. Each canonical state is expanded at most once. Thus the search terminates; exhausting the queue without a positive path implies nontriviality. $\square$

3.2 Actual complexity, and the input-size distinction

Every grid decomposes into two perfect matchings of its corner incidence graph, so it is represented by some ordered pair of permutations. Therefore the number of grids of size k is at most $(k!)^2$. Let

$$S(g) = \sum_{k=2}^g (k!)^2 \leq g(g!)^2.$$

There are $O(g)$ elementary moves at a state. The straightforward optimized successor computation takes polynomial work in g per state; with ordinary hash-table lookup, $O(g^3)$ small-integer operations suffice for its dominant canonicalization work. Even a conservative implementation model allowing quadratic overhead in $S(g)$ still gives

$$T(g) = 2^{O(g \log g)}.$$

State storage is also bounded by this exponential class. This is an upper bound, not an assertion that every input attains it.

The grid size g is *not* the crossing number n . There are at most g^2 crossings, but a large grid may have very few crossings. Its written input has $\Theta(g \log g)$ bits even when its crossing count is zero. Thus substituting n for g would be unjustified. In particular, neither fast runs on examples nor quotienting by cyclic shifts turns this algorithm into $n^{O(\log n)}$ recognition.

4 Exact determinant obstruction

Traverse the knot once. Record each crossing twice, distinguishing its overpass from its underpass. Consecutive underpasses delimit the Wirtinger arcs. If crossing i has over-arc o_i and incoming/outgoing under-arcs u_i, v_i , its Fox coloring relation is

$$2x_{o_i} - x_{u_i} - x_{v_i} = 0.$$

Indices can coincide, so matrix entries are accumulated rather than assigned. The absolute determinant of a reduced coloring matrix is the knot determinant [4]. The program deletes the final row and column and computes the result exactly. The no-crossing case and the empty minor both have determinant one.

The implementation uses fraction-free Bareiss elimination. With current pivot p , previous pivot d , and row/column index k , the update is

$$a_{ij} \leftarrow \frac{pa_{ij} - a_{ik}a_{kj}}{d} \quad (i, j > k).$$

A zero pivot causes a row swap; the sign is tracked. Each division is checked for exactness. This routine is tested against a separate recursive Laplace expansion, including singular matrices and pivoting cases. There is no floating-point rounding and no modulus chosen heuristically.

Only a value different from one proves nontriviality. A value of one is inconclusive and passes control to the exact search. The eight-row cyclic grid with $X_i = i$, $O_i = i + 3 \pmod{8}$ supplies a compact regression example: its computed determinant is one, yet it has a one-state closed negative certificate. Neither this fact nor its certificate depends on attaching a knot-table name to the example.

5 Certificates and the verification boundary

Each certificate includes a format version and the original input grid. The verifier can be bound to a separately supplied expected input, preventing the acceptance of a valid certificate for a different diagram.

Positive path. The certificate lists moves applied to successive canonical representatives. The verifier starts from its own canonical representative, checks every move, canonicalizes the result independently, and requires termination at size two. Cyclic shifts are implicit in this specified convention, not unchecked free permutations.

Non-unit determinant. The verifier reconstructs the input's coloring matrix and recomputes the claimed integer. A claimed value of one is rejected as a negative certificate.

Closed state set. The certificate lists a set C of canonical grids. The verifier checks that the initial representative is in C , every state is a knot of size between three and the initial size, and every legal successor belongs to C . It enumerates all candidate exchanges and all potential destabilization corners independently of the search's move generator; each candidate is passed to the checked move application routine.

Proposition 5.1 (Negative certificate soundness). *A verified closed state set proves that the input is nontrivial.*

Proof. By induction on path length, every grid reachable by a legal non-increasing move sequence remains in C . No member has size two. A monotone simplification path of the kind guaranteed for an unknot would therefore be impossible. $\square$

A closed set need not be minimal or consist only of reachable states; closure and inclusion of the root suffice. Certificates may be exponentially large. Polynomial verification in *certificate size* is not a claim of polynomial-size certificates from this search.

The verifier is not formally verified. It shares input validation, the elementary move checker, and determinant arithmetic with the recognizer. It does not share the search traversal, optimized neighbor enumeration, or optimized cyclic canonicalizer. A regression test deliberately replaces the search generator with an empty generator: the resulting false negative certificate is rejected by the verifier.

6 Implemented hierarchy-related components

6.1 Essential patterns on an already known ball

A boundary pattern consists of disjoint circles and trivalent embedded graphs. The slides define essentialness by discs meeting the pattern at most three times and bounding an empty disc, an arc, or a tripod on the boundary [1, pp. 24–26]. The implemented polynomial test uses the dual-graph criterion of [2, Proposition 2.6]: apart from the empty pattern and a single circle, the pattern must be connected and its spherical dual must be simple and 4-connected, or K_3 or K_4 .

The input records a cyclic order of three darts at each vertex, with the fixed edge involution $\alpha(d) = dxor1$. If ρ is the vertex rotation permutation, the cycles of $\rho\alpha$ are faces. The implementation checks every dart and tests

$$V - E + F = 2$$

for each connected rotation-system component, thereby rejecting a positive-genus embedding. Circles with no vertices are counted separately.

For a connected graph, faces become dual vertices. Each primal edge determines a dual edge. A loop or parallel pair is an immediate obstruction. After the K_3, K_4 special cases, the program enumerates sets of at most three dual vertices and tests whether removal disconnects the dual. With F faces and E edges, this direct implementation costs $O(F^3(F + E))$. It returns a combinatorial obstruction, not an explicit embedded disc in an arbitrary three-manifold.

The ambient hypothesis matters: this code assumes the pattern lies on the boundary of a ball already established by other means. It does not infer that an arbitrary manifold with $b_1 = 0$ is a ball.

6.2 Exact rational first cohomology

The input here is the simplicial complex generated by tetrahedra with globally named vertices, not an arbitrary face-pairing triangulation. Orient each edge uv by $u < v$. A cocycle satisfies

$$c(u, v) + c(v, w) - c(u, w) = 0 \quad (u < v < w)$$

on every triangular face.

Choose a spanning forest of the one-skeleton. Every cohomology class has a representative vanishing on its forest edges: integrate the original cochain along the forest to obtain a vertex

potential and subtract its coboundary. This representative is unique modulo the irrelevant additive constant in each component. Consequently, the nullspace of the triangle equations on the non-forest edges is isomorphic to $H^1(-; \mathbb{Q})$.

The program computes that nullspace by exact rational row reduction. Each basis vector is multiplied by the least common multiple of its denominators and then divided by the greatest common divisor of its integer entries. The resulting integral cocycles form a *rational* basis; no integral-lattice basis claim is made. The matrices have entries $0, \pm 1$, so determinant bounds on minors give polynomial bit-size bounds for the rational calculation. This supplies an actual cocycle computation, not a call to an unspecified homology oracle.

6.3 Normal coordinates without sheet expansion

The cocycle-to-surface construction is also the starting point of the surface construction discussed in [2, Theorem 6.2]. Here is the explicit local implementation.

For a tetrahedron (v_0, v_1, v_2, v_3) , set

$$h_0 = 0, \quad h_i = c(v_0, v_i), \quad i = 1, 2, 3.$$

The cocycle equations imply $h_j - h_i = c(v_i, v_j)$. Extend these heights affinely and intersect with all half-integer levels. If $h_{\sigma(0)} \leq \dots \leq h_{\sigma(3)}$, define

$$d_i = h_{\sigma(i+1)} - h_{\sigma(i)}, \quad i = 0, 1, 2.$$

Then d_0 counts triangles about the lowest vertex, d_2 counts triangles about the highest vertex, and d_1 counts quadrilaterals separating the two lowest vertices from the two highest. Zero gaps handle tied heights. At most one quadrilateral type is present.

Across a shared face, two choices of lift differ by an integer constant. Thus the family of half-integer levels induces the same arc counts on that face. The implementation verifies this explicitly. Coordinates are stored in order

$$T_0, T_1, T_2, T_3, Q_{01|23}, Q_{02|13}, Q_{03|12}.$$

Coefficients remain binary integers: a multiplicity such as 10^{1000} is one integer, not 10^{1000} allocated polygons.

The module checks the cocycle, duplicate tetrahedra, face incidence, and normal matching. It does *not* certify manifoldness or orientability, extract a connected surface component, or cut the manifold. Those qualifications prevent this local construction from being mistaken for the full surface-selection operation in a hierarchy.

A reproducible example triangulates $S^1 \times D^2$ as three triangular prisms, each split into three tetrahedra. Its computed rational H^1 has dimension one. An explicit cocycle has period one around the circle. Scaling it by 10^{500} scales the normal coordinates without expanding the resulting sheets.

6.4 Correct potential arithmetic

For nonnegative complexity digits $a_i \leq G$, pad a hierarchy to length L and define

$$\Phi(a_1, \dots, a_L) = \sum_{i=1}^L a_i (G+1)^{L-i}.$$

The base must be $G+1$ when the digit G is allowed. A schematic “base G ” reading of the slides would not suffice. This implementation uses the base- $(G+1)$ formulation, as does [2, Section 9].

If the first changed digit drops by at least one at position j , the largest possible increase in the suffix is

$$G \sum_{r=0}^{L-j-1} (G+1)^r = (G+1)^{L-j} - 1.$$

It is smaller than the decrease $(G+1)^{L-j}$ of that digit. Therefore Φ drops by at least one. There can be at most $(G+1)^L - 1$ such decreases. If at most L extension operations occur between them, the corresponding iteration bound is $L(G+1)^L$.

HierarchyPotential checks digit bounds, depth, unchanged prefixes, and strict decrease. These arithmetic checks do not prove that any geometric hierarchy constructed from a knot actually satisfies the assumed bounds.

7 What a quasipolynomial integration would still require

Theorem 7.1 (Conditional cost calculation). *Suppose a complete hierarchy implementation has, in terms of the original crossing number n , the following uniform bounds: complexity digits $G \leq C(n+2)^a$; depth $L \leq b \log_2(n+2) + c$; at most polynomially many outer restarts; and polynomial bit-cost for each charged operation, including updates to all compressed representations. Suppose also that the simplification and extension operations satisfy the potential conditions above. Then its total cost is $n^{O(\log n)}$, with the usual adjustment for $n < 2$.*

Proof. For $m = n + 2$, the logarithm of $(G+1)^L$ is

$$L \log_2(G+1) \leq ab(\log_2 m)^2 + O(\log m).$$

The factor L , polynomially many restarts, and polynomial work per operation add only $O(\log m)$ and $O(\log \log m)$ to the logarithm. Exponentiating gives the claimed class. $\square$

This conditional theorem is not instantiated by the code. The missing constructions are substantive and individually identifiable.

Initial geometric state. One needs a layered handle structure of the knot exterior, a generalized Heegaard splitting and its Morse function, and the ambient embedding information used by the generalized Seifert construction. A grid by itself is not this state.

Compressed cuts with provenance. A cut must update attachment maps, component structure, boundary pattern, and labels identifying the surfaces from which boundary pieces arose. The encoding must remain controlled in bit size under repeated operations. Local normal coordinates and face matching are insufficient. No Agol–Hass–Thurston orbit machinery or compressed cutting implementation is supplied here.

Hierarchical multisurfaces. The construction must produce appropriately independent relative homology classes while controlling the relevant *pattern complexity*, not merely the ordinary genus. Compression and pattern-compression must preserve exactly the later data that the multisurface argument allows. The slides themselves warn that their “genus” notation can stand for a related complexity [1, p. 54].

Cheeger regions and weak reduction. The numerical condition on pages 101–108 of the slides is

$$g(\partial M_i) \leq \frac{1}{3} \min\{g(H_i), g(H) - g(H_i)\}.$$

Testing a numerical inequality is not an implementation of the subsequent geometric modification. One must construct the new splitting, perform the necessary weak reduction and Haken-lemma steps, preserve the invariants, and bound the cumulative cost of restarts. Otherwise logarithmic depth has not been established for the executable process.

Final topology and witnesses. The terminal pattern test must be applied only after the relevant ambient manifolds have been justified as balls. A violating cycle must be converted into the specific disc needed by the earlier hierarchy; the disc must then be classified and transported through the stored geometric data. An abstract dual separator alone is not that entire operation.

No function for these missing transformations is exported as though it were implemented. In particular, the actual recognizer never invokes a stubbed `USE CHEEGER REGION` routine and never relies on guessed values of G, L .

8 Validation and observed results

The standard-library unit suite contains 40 tests and passed on Python 3.13.5. It covers malformed input, link rejection, cyclic quotienting, all inverse stabilization positions on selected diagrams, randomized move invariants, certificate tampering, independent move enumeration, determinant arithmetic, spherical embeddings, exact cohomology, normal matching, huge multiplicities, and potential decrease under maximal suffix resets. Python 3.9 grammar compatibility is checked separately; execution was not tested on every supported version or operating system.

8.1 Exhaustive small grids

All connected unoriented labelled grids of sizes two through six were generated. The count at size g is $g!(g-1)!/2$: choose one matching and a single relative cycle, then forget which matching was first. A canonical representative of each cyclic-shift orbit was solved *without* the determinant shortcut. Every certificate was replayed; non-unit determinants were also checked against the answers. Table 1 records the actual run. This is exhaustive coverage of these finite input classes,

g	Labelled grids	Shift orbits	Unknot orbits	Knot orbits	Max. states
2	1	1	1	0	1
3	6	2	2	0	2
4	72	6	6	0	4
5	1,440	64	62	2	13
6	43,200	1,212	1,185	27	63
Total	44,719	1,285	1,256	29	—

Table 1: Exhaustive validation through grid size six. “Max. states” is the largest number retained in an individual recognition search at that size.

not 44,719 independent topology-oracle comparisons. Positive paths and negative closed sets are checked by the verifier and ultimately rely on the mathematical move theorem and the checked elementary rules.

8.2 Larger examples and limits

With the determinant shortcut disabled, the bundled scrambled nine-row unknot was reduced using a ten-move certificate after retaining 651 states. The scrambled seven-row trefoil produced a closed set of 227 states. A scrambled nine-row determinant-one nontrivial knot produced a closed set of 173 states. Every certificate was verified. The corresponding unscrambled determinant-one example has a one-state closed certificate. Exact timings and input crossing counts are in `benchmark-results.json`; the test log and exhaustive results are also supplied.

The same scrambled unknot, run with a one-state cap, correctly returns UNKNOWN. Benchmarks are small, constructed examples, not a representative performance study and not evidence for a quasipolynomial asymptotic bound. No large Goeritz or Haken diagram was reconstructed from an illustration in the slides, so no such benchmark is claimed.

9 Running and extending the archive

From the extracted directory, no installation or external package is needed:

```
python -m unknot recognize examples/scrambled_unknot.json --certificate proof.json
python -m unknot verify proof.json --input examples/scrambled_unknot.json
python -m unknot pattern examples/pattern_triangular_prism.json
python -m unknot normal examples/solid_torus.json
python -m unittest discover -s tests -v
python -m tools.validate_exhaustive --max-size 6
python -m tools.benchmark
```

Use `--max-states`, `--timeout`, and `--no-certificate` for controlled experiments. A timeout is soft: a currently executing operation is not preempted. A certificate requested for an unknown result is not written; an existing file of that name is left unchanged and a warning is issued.

The algorithmic entry points are separated by module: `grid.py` for the representation and move kernel; `recognize.py` for search and verification; `determinant.py` for the obstruction; `pattern.py`, `cohomology.py`, and `normal.py` for the hierarchy primitives; and `potential.py` for the conditional counting argument. Future geometric work can use the latter modules without pretending that the present search already has the desired bound.

References

- [1] Marc Lackenby. *Unknot recognition in quasi-polynomial time*. February 2021 lecture slides, 109 pages. Supplied as `quasipolynomial-talk.pdf`.
- [2] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. 25 July 2026. arXiv:2607.23350v1. <https://arxiv.org/html/2607.23350v1>.
- [3] Ivan Dynnikov. *Arc-presentations of links. Monotonic simplification*. arXiv:math/0208153. In particular, elementary moves and Proposition 6. <https://arxiv.org/pdf/math/0208153>.
- [4] Sudipta Kolay. *Knot Colorings: Coloring and Goeritz matrices*. 2019. arXiv:1910.08044v1. <https://arxiv.org/pdf/1910.08044>.